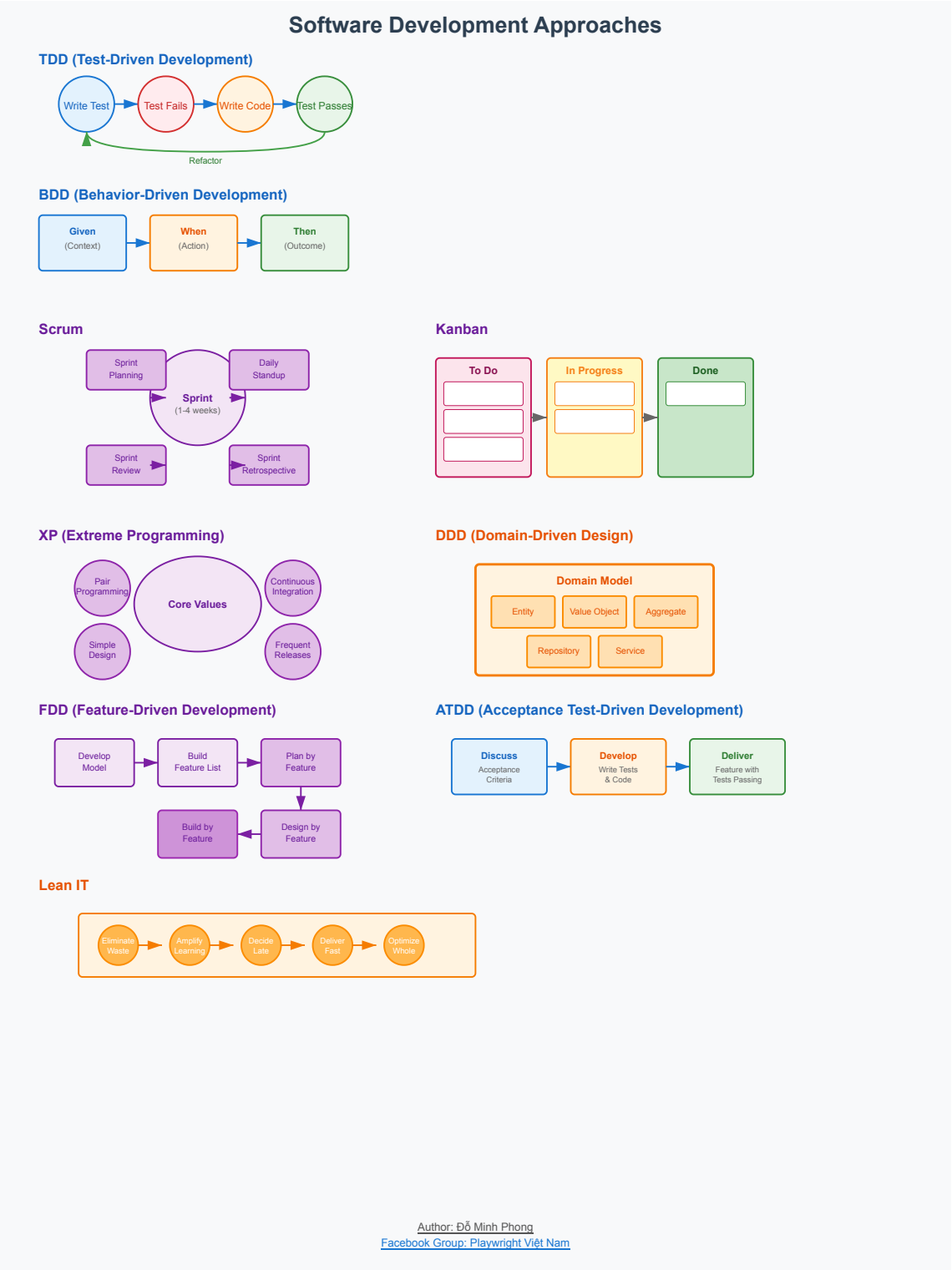
2.1. Kiểm thử trong bối cảnh Vòng đời phát triển phần mềm (SDLC)

SDLC (Software Development Life Cycle) là mô hình mô tả quy trình phát triển phần mềm ở mức độ cao, xác định cách thức các giai đoạn phát triển liên kết với nhau về mặt logic và thời gian.

Loại mô hình	Ví dụ	Đặc điểm
Tuần tự (Sequential)	Waterfall, V-Model	Các giai đoạn diễn ra theo thứ tự, không quay lại
Lặp (Iterative)	Spiral, Prototyping	Lặp lại các chu kỳ nhỏ, cải tiến dần
Tăng trưởng (Incremental)	Unified Process	Phát triển từng phần, tích hợp dần

Một số hoạt động trong quy trình phát triển phần mềm cũng có thể được mô tả chi tiết hơn bởi các phương pháp phát triển phần mềm và các thực hành Agile. Các ví dụ bao gồm: phát triển hướng kiểm thử chấp nhận (ATDD), phát triển hướng hành vi (BDD), thiết kế hướng miền (DDD), lập trình cực hạn (XP), phát triển hướng tính năng (FDD), Kanban, Lean IT, Scrum và phát triển hướng kiểm thử (TDD).

- **Test-first approaches:** TDD, ATDD, BDD
- **Agile practices:** Scrum, Kanban, XP (Extreme Programming), FDD (Feature-Driven Development)
- **Other:** DDD (Domain-Driven Design), Lean IT



2.1.1. Tác động của Vòng đời phát triển phần mềm đối với kiểm thử

Kiểm thử phải được điều chỉnh phù hợp với SDLC để thành công. Việc lựa chọn SDLC có tác động đến:

Yếu tố	Ảnh hưởng như thế nào?

1️⃣ **Phạm vi & Thời điểm**	Khi nào test? Test mức độ nào? (Component/System/Acceptance)
2️⃣ **Tài liệu kiểm thử**	Chi tiết đến đâu? (Agile: nhẹ, Waterfall: chi tiết)
3️⃣ **Kỹ thuật kiểm thử**	Dùng kỹ thuật nào? (Formal vs Experience-based)
4️⃣ **Mức độ tự động hóa**	Tự động bao nhiêu %? (Agile: cao, Waterfall: thấp hơn)
5️⃣ **Vai trò người kiểm thử**	Tester làm gì? (Agile: đa năng, Waterfall: chuyên môn hóa)

Trong các **mô hình phát triển tuần tự**, ở các giai đoạn đầu, người kiểm thử thường tham gia vào việc xem xét yêu cầu, phân tích kiểm thử và thiết kế kiểm thử. Mã nguồn thực thi thường được tạo ra ở các giai đoạn sau, vì vậy thông thường kiểm thử động không thể thực hiện sớm trong SDLC.

Trong một số **mô hình phát triển lặp và tăng trưởng**, mỗi lần lặp được giả định là sẽ bàn giao một bản mẫu hoạt động được hoặc một phần tăng trưởng của sản phẩm. Điều này có nghĩa là trong mỗi lần lặp, cả kiểm thử tĩnh và kiểm thử động đều có thể được thực hiện ở tất cả các mức độ kiểm thử. Việc bàn giao thường xuyên các phần tăng trưởng đòi hỏi phản hồi nhanh và kiểm thử hồi quy trên diện rộng.

Phát triển phần mềm Agile giả định rằng sự thay đổi có thể xảy ra trong suốt dự án. Do đó, các dự án Agile ưu tiên tài liệu sản phẩm công việc nhẹ nhàng và tự động hóa kiểm thử sâu rộng để giúp việc kiểm thử hồi quy dễ dàng hơn. Ngoài ra, hầu hết việc kiểm thử thủ công có xu hướng được thực hiện bằng các kỹ thuật kiểm thử dựa trên kinh nghiệm mà không đòi hỏi phân tích và thiết kế kiểm thử sâu rộng trước đó.

📊 So sánh kiểm thử theo các mô hình SDLC:

Đặc điểm	Mô hình Tuần tự (Waterfall)	Mô hình Lặp/Tăng trưởng	Agile
Khi nào test?	Cuối dự án, sau khi code xong	Cuối mỗi iteration	Liên tục trong mỗi sprint
Test ở giai đoạn đầu	Review tài liệu, phân tích test	Static + Dynamic testing	Static + Dynamic testing
Dynamic testing	Muộn (code xong mới test)	Sớm hơn (mỗi iteration)	Rất sớm (TDD/ATDD/BDD)
Tài liệu test	Chi tiết, đầy đủ	Vừa phải	Nhẹ, tối giản
Regression testing	Ít, vì ít thay đổi	Nhiều, mỗi iteration	Rất nhiều, automation cao
Test automation	Thấp-Trung bình	Trung bình-Cao	Cao (bắt buộc)

2.1.2. Vòng đời phát triển phần mềm và các best practices

Bất kể dùng SDLC nào, **4 thực hành này luôn đúng:**

✅ 1. Mỗi hoạt động phát triển → Có hoạt động kiểm thử tương ứng

```
Yêu cầu (Requirements)     → Review yêu cầu
Thiết kế (Design)          → Review thiết kế
Lập trình (Coding)         → Unit testing, Static analysis
```

```
Tích hợp (Integration)     → Integration testing
Triển khai (Deployment)    → Acceptance testing
```

✅ 2. Mỗi mức độ kiểm thử có mục tiêu riêng biệt

- **Component Testing:** Kiểm tra logic từng module
- **Integration Testing:** Kiểm tra giao tiếp giữa các module
- **System Testing:** Kiểm tra toàn bộ hệ thống
- **Acceptance Testing:** Kiểm tra đáp ứng yêu cầu khách hàng

➡️ **Tránh dư thừa:** Không test lại cùng một thứ ở nhiều mức độ

✅ 3. Phân tích & Thiết kế test bắt đầu sớm

```
❌ SAI:  Design → Code → [BẮT ĐẦU viết test case]
✅ ĐÚNG: Design → [BẮT ĐẦU viết test case song song] → Code
```

➡️ Tuân thủ nguyên tắc **"Testing Early"**

✅ 4. Tester tham gia review ngay từ khi có bản nháp

```
📄 Draft Requirements sẵn → Tester review ngay
📐 Draft Design sẵn       → Tester review ngay
```

➡️ Phát hiện lỗi sớm, hỗ trợ **Shift Left**

2.1.3. Kiểm thử đóng vai trò dẫn dắt sự phát triển phần mềm (Test-First Approaches)

🔴 TDD (Test-Driven Development) - Phát triển hướng kiểm thử

Quy trình:

```
1. Viết test case (chưa có code) → Test FAIL (đỏ) ❌
2. Viết code tối thiểu để pass test → Test PASS (xanh) ✅
3. Refactor code (tối ưu hóa) → Test vẫn PASS ✅
4. Lặp lại chu kỳ
```

Đối tượng: Lập trình viên (developer)
Mức độ: Component/Unit level
Ví dụ: Viết test cho hàm `calculateTotal()` trước, sau đó mới viết code hàm

🟢 ATDD (Acceptance Test-Driven Development) - Phát triển hướng kiểm thử chấp nhận

Quy trình:

```
1. Xác định tiêu chí chấp nhận (Acceptance Criteria)
2. Viết test từ tiêu chí này
3. Phát triển tính năng để pass test
```

Đối tượng: Cả team (BA, Dev, Tester)
Mức độ: System/Acceptance level
Ví dụ:

- Tiêu chí: "User phải đăng nhập thành công với email hợp lệ"

- Viết test: Nhập email hợp lệ → Kiểm tra chuyển đến trang Home
- Dev viết code để pass test này

🔵 BDD (Behavior-Driven Development) - Phát triển hướng hành vi

Quy trình:

```
1. Mô tả hành vi bằng ngôn ngữ tự nhiên (Given/When/Then)
2. Tự động chuyển đổi thành test thực thi
3. Phát triển code để pass test
```

Đối tượng: Cả team + Stakeholders (dễ hiểu)
Mức độ: System/Acceptance level
Ví dụ:

```
Given user đang ở trang đăng nhập
When user nhập email "test@example.com" và password "123456"
And user nhấn nút "Đăng nhập"
Then user được chuyển đến trang Home
And hiển thị thông báo "Đăng nhập thành công"
```

📋 So sánh TDD vs ATDD vs BDD:

Tiêu chí	TDD	ATDD	BDD
Người viết test	Developer	BA/Tester/Developer	Cả team + Stakeholders
Ngôn ngữ	Code (Unit test)	Code/Tools	Ngôn ngữ tự nhiên
Mức độ	Component	System/Acceptance	System/Acceptance
Mục đích	Code đúng logic	Đáp ứng yêu cầu	Đáp ứng hành vi mong đợi
Ví dụ tool	JUnit, NUnit	FitNesse, Robot Framework	Cucumber, SpecFlow

🎯 **Điểm chung của cả 3:**

- ✅ Test được viết **trước** code
- ✅ Tuân thủ nguyên tắc **"Testing Early"**
- ✅ Hỗ trợ **Shift Left**
- ✅ Test được giữ lại làm **Automated Regression Test**

2.1.4. DevOps và Kiểm thử

DevOps là một phương pháp tiếp cận tổ chức nhằm tạo ra sự cộng hưởng bằng cách để bộ phận phát triển (bao gồm cả kiểm thử) và bộ phận vận hành làm việc cùng nhau để đạt được các mục tiêu chung. DevOps thúc đẩy quyền tự chủ của đội ngũ, phản hồi nhanh, các chuỗi công cụ tích hợp và các thực hành kỹ thuật như tích hợp liên tục (CI) và bàn giao liên tục (CD).

📘 DevOps là gì?

DevOps = Development + Operations làm việc cùng nhau

Mục tiêu: Giao phần mềm nhanh hơn, chất lượng cao hơn

Thực hành chính:

- **CI (Continuous Integration):** Tích hợp code liên tục
- **CD (Continuous Delivery/Deployment):** Giao hàng/Triển khai liên tục
- **Automation:** Tự động hóa mọi thứ có thể

✅ Lợi ích của DevOps cho Kiểm thử:

Lợi ích	Giải thích
1️⃣ **Phản hồi nhanh về chất lượng**	Code commit → Chạy test tự động → Biết kết quả ngay (trong vài phút)
2️⃣ **Thúc đẩy Shift Left**	CI bắt dev phải viết code chất lượng + unit test trước khi commit
3️⃣ **Môi trường test ổn định**	CI/CD tự động setup môi trường → Không còn "works on my machine"
4️⃣ **Tăng khả năng hiển thị Non-functional**	Dashboard hiển thị performance, reliability metrics liên tục
5️⃣ **Giảm manual testing lặp lại**	Automation xử lý regression test → Tester tập trung exploratory
6️⃣ **Giảm rủi ro regression**	Test tự động chạy mỗi lần commit → Phát hiện lỗi ngay lập tức

⚠️ Thách thức của DevOps:

Thách thức	Giải pháp
Phải setup CI/CD pipeline	Đầu tư thời gian ban đầu, dùng Jenkins/GitLab CI/GitHub Actions
Duy trì CI/CD tools	Cần DevOps engineer hoặc đào tạo team
Test automation tốn resources	Đầu tư dần dần, ưu tiên test quan trọng trước
Vẫn cần manual testing	Kết hợp automation + exploratory testing

💡 Ví dụ quy trình DevOps Testing:

```
1. Dev viết code + unit test → Commit lên Git
2. CI server (Jenkins) tự động:
   ✓ Build code
   ✓ Chạy unit tests
   ✓ Static analysis (SonarQube)
   ✓ Integration tests
3. Nếu PASS → Tự động deploy lên môi trường Test
4. Automation tests chạy trên môi trường Test
5. Nếu PASS → Tự động deploy lên Staging
6. Manual exploratory testing trên Staging
7. Nếu OK → Deploy lên Production (CD)
```

2.1.5. Dịch chuyển sang trái (Shift Left)

Nguyên tắc kiểm thử sớm đôi khi được gọi là **dịch chuyển sang trái** vì đây là cách tiếp cận mà việc kiểm thử được thực hiện sớm hơn trong SDLC. Dịch chuyển sang trái gợi ý rằng kiểm thử nên được thực hiện sớm (ví dụ: không đợi mã được triển khai xong hoặc các thành phần được tích hợp xong), nhưng điều đó **không có nghĩa** là việc kiểm thử ở các giai đoạn sau của SDLC bị bỏ qua.

Shift Left = Thực hiện kiểm thử **sớm hơn** trong SDLC

```
Timeline dự án (từ trái sang phải):
[Yêu cầu] → [Thiết kế] → [Code] → [Test] → [Deploy]
    ↑          ↑          ↑         ↑
    Test      Test       Test      Test

← Dịch chuyển các hoạt động test về BÊN TRÁI (sớm hơn)
```

❗ Lưu ý quan trọng:

Shift Left ≠ Bỏ qua test ở giai đoạn sau

✅ ĐÚNG: Test sớm hơn + Test đầy đủ ở tất cả giai đoạn
❌ SAI: Chỉ test sớm, bỏ qua system test và acceptance test

📋 5 thực hành tốt để Shift Left:

STT	Thực hành	Giải thích	Ví dụ
1️⃣	**Review đặc tả từ góc độ tester**	Tìm mơ hồ, thiếu sót, mâu thuẫn	Review User Story: "Login nhanh" → Hỏi: Nhanh là bao nhiêu giây?
2️⃣	**Viết test case trước khi code**	TDD/ATDD/BDD	Viết test "User login với email hợp lệ" trước khi dev viết code
3️⃣	**Dùng CI/CD**	Phản hồi nhanh, unit test tự động	Mỗi lần commit → Chạy unit test ngay
4️⃣	**Static analysis trước Dynamic test**	Tìm lỗi cú pháp, code smell trước	Chạy SonarQube trước khi chạy integration test
5️⃣	**Non-functional test từ mức Component**	Test performance ngay ở unit level	Test performance của hàm `calculateTotal()` xử lý 10,000 items

💰 Chi phí vs Lợi ích:

```
Chi phí:
├─ Giai đoạn đầu: ↑ Tăng (đầu tư thời gian review, viết test sớm)
└─ Giai đoạn sau:  ↓ Giảm (ít bug, sửa dễ hơn)

Tổng thể: 📉 Tiết kiệm
```

Tại sao lại tiết kiệm?

Giai đoạn phát hiện lỗi	Chi phí sửa	Lý do

Yêu cầu	1x	Chỉ cần sửa tài liệu
Thiết kế	5x	Sửa tài liệu + thiết kế lại
Code	10x	Sửa code + test lại
Test	15x	Sửa code + test lại + regression
Production	**100x**	Sửa khẩn cấp + ảnh hưởng khách hàng + mất uy tín

🍲 Ví dụ dễ nhớ: Nấu súp

```
❌ KHÔNG Shift Left:
   Nấu súp 3 tiếng → Nếm thử → Phát hiện quá mặn
   → Khó sửa: Phải pha thêm nước, rau củ, ninh lại...

✅ CÓ Shift Left:
   Vừa cho nguyên liệu → Nếm thử ngay
   → Dễ sửa: Thêm chút muối/đường là xong
```

Bài học: Phát hiện lỗi càng sớm → Sửa càng dễ, rẻ

2.1.6. Các cuộc họp cải tiến (Retrospectives) và Cải tiến quy trình

Các cuộc họp cải tiến thường được tổ chức vào cuối dự án, cuối đợt lặp, tại một mốc phát hành hoặc khi cần thiết. Trong các cuộc họp này, các thành viên (không chỉ người kiểm thử mà cả lập trình viên, kiến trúc sư, chủ sản phẩm...) thảo luận về: **Cái gì đã thành công (cần giữ lại)? Cái gì không thành công (cần cải thiện)? Làm thế nào để áp dụng các cải tiến đó vào tương lai?**.

📅 Retrospective là gì?

Retrospective (Họp cải tiến) = Cuộc họp để **nhìn lại và cải thiện**

Khi nào họp?

- Cuối dự án
- Cuối mỗi iteration/sprint
- Sau mỗi release
- Khi cần thiết (có vấn đề lớn)

👥 Ai tham gia?

TOÀN BỘ team, không chỉ tester:

- Developers
- Testers
- Architects
- Product Owner
- Scrum Master
- BA, UX...

💬 3 câu hỏi chính:

Câu hỏi	Mục đích	Ví dụ câu trả lời

1️⃣ **Cái gì đã tốt?**	Giữ lại những gì hiệu quả	"Automation giúp phát hiện lỗi nhanh"
2️⃣ **Cái gì chưa tốt?**	Xác định vấn đề	"Test environment hay bị crash"
3️⃣ **Làm thế nào cải thiện?**	Hành động cụ thể	"Nâng cấp server test environment"

✅ Lợi ích cho Kiểm thử:

Lợi ích	Giải thích	Ví dụ
1️⃣ **Tăng hiệu quả kiểm thử**	Tìm cách test nhanh hơn, ít lỗi hơn	"Áp dụng risk-based testing thay vì test toàn bộ"
2️⃣ **Tăng chất lượng testware**	Test case rõ ràng, dễ maintain	"Refactor test scripts, loại bỏ test cases trùng lặp"
3️⃣ **Team gắn kết & học hỏi**	Chia sẻ kiến thức, kinh nghiệm	"Junior tester học cách viết test case từ Senior"
4️⃣ **Cải thiện test basis**	Tài liệu requirements tốt hơn	"Yêu cầu BA viết acceptance criteria rõ ràng hơn"
5️⃣ **Hợp tác Dev-Test tốt hơn**	Giảm xung đột, tăng hiểu biết lẫn nhau	"Dev giúp tester setup môi trường test"

🎯 Ví dụ Retrospective thực tế:

Sprint vừa rồi:

- ✅ Tốt: Automation coverage tăng từ 60% → 75%
- ❌ Chưa tốt: Nhiều bug regression vì dev không chạy full regression test trước khi merge code
- 🔧 Cải thiện:
  - Setup CI/CD để chạy tự động regression test mỗi lần merge
  - Tổ chức workshop hướng dẫn dev cách chạy regression test locally

📝 Template Retrospective cho Kiểm thử:

```
## Sprint X Retrospective - Testing Perspective

### ✅ What went well?
- [Item 1]
- [Item 2]

### ❌ What didn't go well?
- [Item 1]
- [Item 2]

### 🔧 Action items for next sprint:
1. [Action 1] - Owner: [Name] - Deadline: [Date]
2. [Action 2] - Owner: [Name] - Deadline: [Date]
```

Một số ghi chú

- Tài liệu dịch từ sách gốc ISTQB Foudation v4.0.1 sang tiếng Việt, nhằm mục đích giảm rào cản tiếp cận các kiến thức về testing tới cộng đồng tester Việt Nam nói chung và các anh chị em muốn tìm hiểu về testing nói riêng
- Tài liệu cố gắng dịch nhiều nhất các từ tiếng Việt để bạn đọc không phải tra từ điển trong quá trình đọc (VD: defects dịch là khuyết tật phần mềm).
- Dự án phi lợi nhuận, bạn có thể thoải mái sử dụng bản dịch, chia sẻ và sửa đổi nếu cần thiết.
- Có góp ý cho dự án, bạn có thể submit góp ý qua link này nha: https://go.betterbytesvn.com/sharing-documentation-feedback.
- Bản dịch được thực hiện bởi tác giả Đỗ Minh Phong. Bạn có thể gửi lời cảm ơn/feedback/ donate tới tác giả thông qua các hình thức:
    - Đăng ký kênh Youtube: https://www.youtube.com/@hoctest_com
    - Follow Fanpage: https://www.facebook.com/hoctest/
    - Tham gia group Playwright Việt Nam trên Facebook: https://www.facebook.com/groups/playwright.automation.test
    - Donate cho tác giả: https://academy.betterbytesvn.com/donate-cho-chung-minh/
    - Facebook cá nhân tác giả: https://www.facebook.com/dominhphong.18/

Xin chân thành cảm ơn bạn, vì đã quan tâm tới tài liệu ^^.